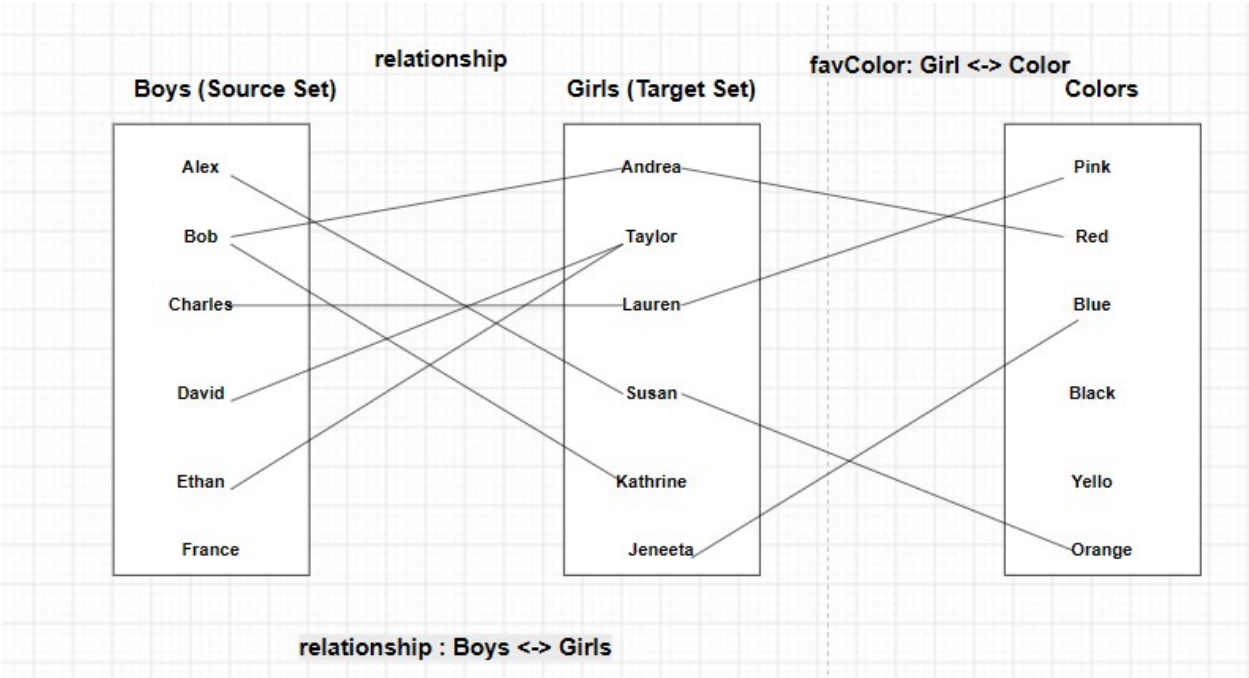
Relations

- Relations are used to connect elements of 2 SETS



- A relation can have one-to-one, one-to-many, many-to-one, and many-to-many relation.

1. Source Set

The set from which relation begins is called the source set. Always represented in the left side.

Ex- **Boys Set**

2. Target Set

The set where the relation end is called the target set. Always represented in right side.

Ex- Girl Set

3. Defining Relation in Abstract Machine

relationName : Source Set <-> Target Set

ex- relationship: Boys <-> Girls

4. Maplets

Each element in a relation is called a maplet. Maplets are in pairs. Maplets can be represented in 2 ways

1. (Alex, Susan)- Maplet (Not recommended. Atelier does not recognize it as maplet)
2. Alex |-> Susan- Maplet (Recommended)

5. Domain Elements

Domain elements include all the elements of **source set** that is having a pair.

dom(relationship)= {Alex, Bob, Charles, David, Ethan}

France is not a domain element as it does not have a pair

6. Range Element

Range elements include all the elements of **target set** that have a pair.

ran(relationship)= {Andrea, Taylor, Lauren, Susan, Kathrine}

Janeeta is not a range element as it does not have a pair

7. Projections

Used to get either domain or range element separately of **a particular maplet.**

1. Projection 1(prj1)- Gives the domain element of the maplet.

Ex- prj1(Alex, Susan) = Alex

prj1(Charles, Lauren) = Charles

2. Projection 2(prj2)- Gives the range element of the maplet.

Ex- prj2(Alex, Susan) = Susan

prj2(Charles, Lauren) = Lauren

8. Inverse of Relation (~)

Inverse of relation swaps the source set and the target set

relationship= Boys <-> Girls

relationship~ = Girls <-> Boys = {Susan |-> Alex, Andrea |-> Bob, Kathrine |-> Bob, Lauren |-> Charles, Taylor |-> David, Taylor |-> Ethan}

9. Identity Relation

Element pairing to itself is called as identity relation

id(Boys)= {Alex |->Alex, Bob |-> Bob, Charles |-> Charles, David |-> David, Ethan |-> Ethan, France |-> France}

id(Girls)= {Andrea |-> Andrea, Taylor |-> Taylor, Lauren |-> Lauren, Susan |-> Susan, Kathrine |-> Kathrine, Jeneeta |-> Jeneeta}

10. Relational Lookup/ Relational Image

Relational Lookup gives all the range elements associated with a specific domain element.

relationship[{Alex] = {Susan}

relationship [{Bob, Charles}] = {Andrea, Kathrine, Lauren}

11. Domain Restriction

Gives all the maplets associated with a specific domain element

{Alex} < | relationship = {Alex |-> Susan}

{Bob, Ethan} < | relationship= {Bob |-> Andrea, Bob |-> Kathrine, Ethan |-> Taylor}

12. Range Restriction

Gives all the maplets associated with a specific range element

relationship | > {Andrea} = {Alex |-> Andrea}

relationship |> {Taylor} = {David |-> Taylor, Ethan |-> Taylor}

13. Domain Anti-Restriction / Domain substraction

Gives all maplets **except** for the maplet associated with the specific domain element

{Bob} << | relationship = {Alex |-> Susan, Charles |-> Lauren, David |-> Taylor, Ethan |-> Taylor}

{David, Ethan} << | relationship= {Alex |-> Susan, Bob |-> Andrea, Bob |-> Kathrine, Charles |-> Lauren}

14. Range Anti-Restriction/ Range

Gives all maplets except for the maplet associated with the specific range element.

relationship |>> {Taylor}= {Alex |-> Susan, Bob |-> Andrea, Bob |-> Kathrine, Charles |-> Lauren}

relationship | >> {Taylor, Lauren} = {Alex |-> Susan, Bob |-> Andrea, Bob |-> Kathrine}

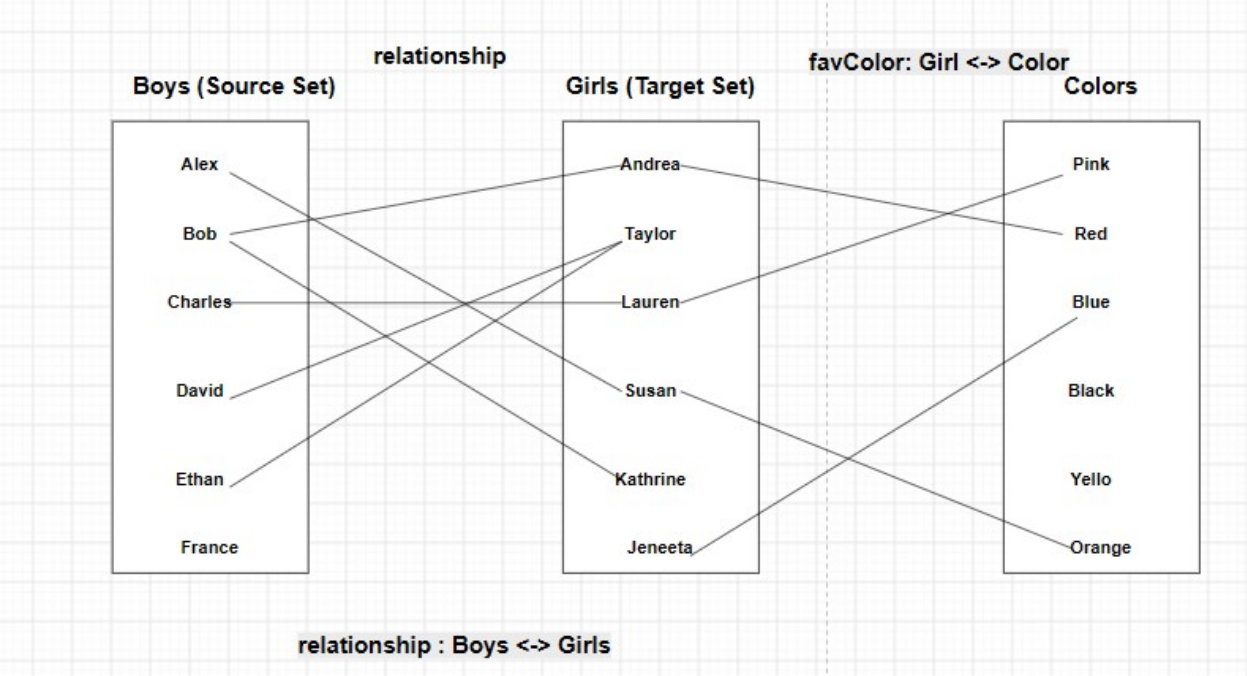
15.Relational Override

Relational Override is used to add new element or update an existing element.

relationship <+ {Bob |-> Janeeta} = {Alex |-> Susan, Bob |-> Janeeta, Charles |-> Lauren, David |-> Taylor, Ethan |-> Taylor}

relationship <+ {France |-> Janeeta}= {Alex |-> Susan, Bob |-> Andrea, Bob |-> Kathrine , Charles |-> Lauren, David |-> Taylor, Ethan |-> Taylor, France |-> Janeeta}

16. Relational Composition



relationship ; favColor = {Alex |-> Orange, Bob |-> Red, Charles |-> Pink}